

ALGORITHM ANALYSIS AND DESIGN

BASIC LEVEL - MASTER DETAILED NOTES

1. Introduction to Algorithm

An algorithm is a step-by-step finite procedure to solve a problem. It must satisfy five properties: Input, Output, Definiteness, Finiteness, and Effectiveness. Algorithms form the foundation of computer programming and problem solving.

2. Characteristics of a Good Algorithm

A good algorithm must be correct, efficient, simple, and scalable. Efficiency is measured in terms of time and space complexity.

3. Time Complexity

Time complexity measures how the running time increases with input size n . We analyze worst-case, average-case, and best-case performance. Examples: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$.

4. Space Complexity

Space complexity measures the memory required by an algorithm. It includes input space and auxiliary space such as variables and recursion stack.

5. Asymptotic Notations

Big-O represents upper bound, Big-Omega represents lower bound, and Big-Theta represents tight bound of growth rate.

6. Lower Bound Concept

Lower bound represents the minimum time required to solve a problem. It helps determine optimality of algorithms.

7. Tractable Problems

Problems solvable in polynomial time ($O(n^k)$) are tractable and practical.

8. Intractable Problems

Problems requiring exponential time ($O(2^n)$, $O(n!)$) are intractable and impractical for large inputs.

9. Class P

Contains decision problems solvable in polynomial time using deterministic algorithms.

10. Class NP

Contains decision problems whose solutions can be verified in polynomial time.

11. NP-Hard Problems

Every problem in NP can be reduced to NP-Hard in polynomial time. NP-Hard problems may not have polynomial-time verification.

12. NP-Complete Problems

Problems that are both in NP and NP-Hard. If any NP-Complete problem is solved in polynomial time, then $P = NP$.

13. Polynomial Time Reduction

A technique used to transform one problem to another in polynomial time to prove NP-Completeness.

14. Approximation Algorithms

Used for NP-Hard optimization problems. They produce near-optimal solutions with guaranteed approximation ratio.

15. Performance Ratio

If C is approximate cost and C^* is optimal cost, approximation ratio = C/C^* (for minimization problems).

16. Bin Packing Problem

Given items and bin capacity, minimize number of bins. Lower bound = $\text{ceil}(\text{total weight} / \text{capacity})$. It is NP-Hard.

17. Graph Coloring

Assign colors to vertices such that adjacent vertices have different colors. Minimum number of colors required is chromatic number.

18. Randomized Algorithms

Algorithms that use randomness to improve expected performance or avoid worst-case inputs.